

Universal RAG System

AI-Powered Document Question & Answer System

Full-Stack Web Application

FastAPI + React + Groq LLM + LangChain

Developer: Karun

May 2026

Table of Contents

- 1. Project Overview
- 2. Tech Stack
- 3. System Architecture
- 4. Backend Structure
- 5. Frontend Structure
- 6. Database Design
- 7. RAG Pipeline
- 8. API Endpoints
- 9. Authentication Flow
- 10. File Processing
- 11. Setup & Installation
- 12. Environment Variables
- 13. Known Limitations & Future Scope

1. Project Overview

Universal RAG System is a full-stack web application that allows users to upload documents (PDF, DOCX, XLSX, PPTX, TXT) and ask questions about their content using AI. The system uses Retrieval-Augmented Generation (RAG) to find relevant document chunks and generate accurate answers using a Large Language Model (Groq LLM).

Key Features

- User authentication (register, login, JWT tokens)
- Multi-format document upload (PDF, DOCX, XLSX, PPTX, TXT)
- Automatic text extraction and chunking using LangChain
- AI-powered Q&A with context-based answers
- Confidence scoring for each answer
- Bookmark important queries
- Document management (list, view, delete)
- Query history with statistics
- Dark-themed responsive UI

2. Tech Stack

Backend

- Python 3.10+
- FastAPI - High-performance async web framework
- SQLAlchemy ORM - Database operations
- SQLite - Lightweight relational database
- Pydantic v2 - Data validation and serialization
- PyJWT - JSON Web Token authentication
- bcrypt - Password hashing (direct, no passlib)
- LangChain - Document loading and text splitting
- Groq API (ChatGroq) - LLM inference (llama-3.3-70b-versatile)
- Sentence Transformers - Local embeddings (all-MiniLM-L6-v2)
- python-multipart - File upload handling

Frontend

- React 18 with Vite build tool
- Tailwind CSS - Utility-first styling
- React Router v6 - Client-side routing
- Axios - HTTP client with interceptors
- react-dropzone - Drag-and-drop file upload
- react-hot-toast - Toast notifications
- lucide-react - Icon library

3. System Architecture

The application follows a client-server architecture with a clear separation between the React frontend and FastAPI backend. Communication happens via REST API calls with JWT token-based authentication.

High-Level Flow

```
User (Browser)
  |
  v
React Frontend (Vite, port 5173)
  | (Axios HTTP calls)
  v
FastAPI Backend (Uvicorn, port 8000)
  |
  +---> SQLite Database (test.db)
  +---> Groq Cloud API (LLM inference)
  +---> Local File Storage (./uploads/)
  +---> Sentence Transformers (local embeddings)
  +---> Pinecone (optional, fails gracefully)
```

Request Flow for Q&A

1. User types question in ChatBox
2. Frontend POST /queries/ask {question, document_id}
3. Backend validates input and user auth
4. RAG Engine searches Embedding table (keyword matching)
5. Top 5 relevant chunks retrieved
6. Context + question sent to Groq LLM
7. LLM generates answer
8. Answer saved to Query table with confidence score
9. Response returned to frontend
10. ChatBox displays answer with confidence bar

4. Backend Structure

Directory Layout

backend/	
main.py	- FastAPI app setup, CORS, router includes
database.py	- SQLAlchemy engine, session, Base
models.py	- ORM models (User, Document, Embedding, Query)
schemas.py	- Pydantic schemas for request/response validation
dependencies.py	- get_current_user dependency (JWT auth)
routes/	
auth.py	- /auth/register, /auth/login, /auth/me
documents.py	- /documents/upload, list, get, delete, stats
queries.py	- /queries/ask, list, get, delete, bookmark, stats
utils/	
security.py	- Password hashing (bcrypt), JWT create/decode
rag_engine.py	- Groq LLM, DB chunk search, answer generation
embeddings.py	- HuggingFace Sentence Transformers wrapper
file_processor.py	- Text extraction (PDF/DOCX/XLSX/PPTX/TXT), chunking, validation

main.py

Entry point of the backend. Creates the FastAPI app instance, configures CORS middleware (allowing localhost:3000 and localhost:5173), includes auth/documents/queries routers, and runs database table creation on startup event.

database.py

Sets up SQLAlchemy with SQLite. Reads DATABASE_URL from environment (.env file), defaults to sqlite:///./test.db. Provides get_db() dependency for session injection.

models.py

Defines 4 SQLAlchemy ORM models:

- User: id, email (unique), name, password_hash, is_active, created_at, last_login
- Document: id, user_id (FK), filename, file_path, file_type, file_size, content, processing_status, is_processed, processing_error, created_at
- Embedding: id, document_id (FK), chunk_text, chunk_index, vector_id, created_at
- Query: id, user_id (FK), document_id (FK, nullable), question, answer, context, confidence, is_bookmarked, created_at

schemas.py

Pydantic v2 schemas for API validation:

- UserCreate: email (EmailStr), name, password (with validator: uppercase+lowercase+number)
- UserLogin: email, password
- UserResponse: id, email, name, created_at, last_login, is_active
- DocumentResponse/DocumentListResponse: document metadata fields
- QueryCreate: question (min 5 chars), optional document_id
- QueryResponse/QueryListResponse: answer, confidence, bookmark status
- Token: access_token, token_type, user (UserResponse)

dependencies.py

Contains `get_current_user()` which uses `OAuth2PasswordBearer` to extract the JWT token from the Authorization header, decodes it using `security.decode_token()`, and fetches the user from the database. Raises 401 if token invalid/expired.

5. Frontend Structure

Directory Layout

```
frontend/
  index.html          - HTML entry point
  vite.config.js      - Vite configuration
  tailwind.config.js  - Tailwind CSS config
  package.json        - Dependencies
  src/
    main.jsx          - React root with BrowserRouter
    App.jsx            - Routes (login, register, dashboard)
    index.css          - Tailwind directives
    context/
      AuthContext.jsx  - Auth state management (token, user, login/register/logout)
    pages/
      LoginPage.jsx    - Login form with email/password
      RegisterPage.jsx - Registration form with password strength indicator
      DashboardPage.jsx - Main app: sidebar + chat/documents/history panels
    components/
      Sidebar.jsx      - Navigation sidebar (Chat, Documents, History, Logout)
      ChatBox.jsx      - Chat interface with document selector, messages, typing indicator
      FileUpload.jsx    - Drag-and-drop file upload with queue management
      DocumentList.jsx  - Document list with stats, status badges, delete/retry
    utils/
      api.js           - Axios instance with token interceptor, API wrappers
```

Key Components

AuthContext.jsx: Manages authentication state using React Context. Stores JWT token and user object in localStorage. On mount, validates token via /auth/me endpoint. Provides login(), register(), logout() functions to all child components.

ChatBox.jsx: Main chat interface. Features document selector dropdown to scope questions to a specific document, message bubbles (blue for user, gray for AI), confidence bar, bookmark toggle, typing indicator with bounce animation, and error display.

FileUpload.jsx: Uses react-dropzone for drag-and-drop. Maintains an upload queue with status tracking (waiting/loading/done/error). Supports PDF, DOCX, XLSX, PPTX, TXT up to 50MB.

DashboardPage.jsx: Main layout with fixed sidebar (w-64) and content area (ml-64). Contains three panels switchable via sidebar: Chat, Documents, and History.

6. Database Design

SQLite database with 4 tables, managed by SQLAlchemy ORM.

Users Table

id	INTEGER	PRIMARY KEY AUTOINCREMENT
email	VARCHAR	UNIQUE NOT NULL
name	VARCHAR	NOT NULL
password_hash	VARCHAR	NOT NULL (bcrypt hashed)
is_active	BOOLEAN	DEFAULT TRUE
created_at	DATETIME	
last_login	DATETIME	NULLABLE

Documents Table

id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	FK -> users.id
filename	VARCHAR	NOT NULL
file_path	VARCHAR	NOT NULL
file_type	VARCHAR	(pdf/docx/xlsx/pptx/txt)
file_size	INTEGER	
content	TEXT	(full extracted text)
processing_status	VARCHAR	(pending/processing/completed/failed)
is_processed	BOOLEAN	DEFAULT FALSE
processing_error	TEXT	NULLABLE
created_at	DATETIME	

Embeddings Table

id	INTEGER	PRIMARY KEY AUTOINCREMENT
document_id	INTEGER	FK -> documents.id
chunk_text	TEXT	NOT NULL
chunk_index	INTEGER	
vector_id	VARCHAR	NULLABLE (for Pinecone)
created_at	DATETIME	

Queries Table

id	INTEGER	PRIMARY KEY AUTOINCREMENT
user_id	INTEGER	FK -> users.id
document_id	INTEGER	FK -> documents.id NULLABLE
question	TEXT	NOT NULL
answer	TEXT	
context	TEXT	
confidence	FLOAT	(0.0 - 1.0)
is_bookmarked	BOOLEAN	DEFAULT FALSE
created_at	DATETIME	

7. RAG Pipeline

The RAG (Retrieval-Augmented Generation) pipeline is the core intelligence of the system. It combines document search with LLM generation to provide accurate, context-based answers.

Document Ingestion

1. User uploads file (e.g., report.pdf)
2. File saved to ./uploads/ with unique name
3. LangChain loader extracts text:
 - PDF -> PyPDFLoader
 - DOCX -> Docx2txtLoader
 - TXT -> TextLoader
4. RecursiveCharacterTextSplitter splits text:
 - chunk_size: 1000 chars
 - chunk_overlap: 200 chars
 - Smart separators: paragraph > newline > sentence > word
5. Each chunk stored in Embeddings table
6. Pinecone vector storage attempted (fails gracefully)
7. Document marked as "completed"

Query Processing

1. User asks question
2. search_db_chunks() performs keyword matching:
 - Splits question into words
 - Counts keyword matches in each chunk
 - Returns top 5 most relevant chunks
3. Context built from matched chunks
4. Groq LLM called with RAG prompt:
 - Model: llama-3.3-70b-versatile
 - Temperature: 0.3
 - Max tokens: 1024
5. Confidence calculated: $\min(1.0, \text{chunks_found} / 5)$
 - No chunks found: 0.3 (general knowledge answer)
6. Answer saved to Queries table

Available LLM Models

- | | |
|------------|--------------------------------------|
| llama3-70b | -> llama-3.3-70b-versatile (default) |
| llama3-8b | -> llama-3.1-8b-instant |
| deepseek | -> deepseek-r1-distill-llama-70b |
| mixtral | -> mixtral-8x7b-32768 |

8. API Endpoints

Authentication (/auth)

POST /auth/register - Create new user account
Body: {email, name, password}
Response: UserResponse

POST /auth/login - Login and get JWT token
Body: {email, password}
Response: {access_token, token_type, user}

GET /auth/me - Get current user info (requires auth)
Response: UserResponse

Documents (/documents)

POST /documents/upload - Upload and process document
Body: multipart/form-data (file)
Response: DocumentResponse (201)

GET /documents/ - List user documents
Params: skip, limit
Response: List[DocumentListResponse]

GET /documents/{id} - Get document details

DELETE /documents/{id} - Delete document + embeddings + file

GET /documents/stats/summary - Document statistics

Queries (/queries)

POST /queries/ask - Ask question (RAG pipeline)
Body: {question, document_id?}
Response: QueryResponse (201)

GET /queries/ - List user queries
Params: skip, limit, bookmarked_only

GET /queries/{id} - Get query details

DELETE /queries/{id} - Delete query

POST /queries/{id}/bookmark - Toggle bookmark

GET /queries/stats/summary - Query statistics

9. Authentication Flow

Registration

1. User fills register form (name, email, password)
2. Frontend POST /auth/register with JSON body
3. Backend validates: email unique, password strong (requires uppercase + lowercase + number, min 8 chars)
4. Password hashed with bcrypt (10 rounds, 72-byte limit)
5. User record created in database
6. Response: user info (no password)

Login

1. User enters email + password
2. Frontend POST /auth/login with JSON body
3. Backend verifies password with bcrypt
4. JWT token created (HS256, 1 hour expiry)
Payload: {sub: user_id, exp: timestamp}
5. Response: {access_token, token_type: "bearer", user}
6. Frontend stores token + user in localStorage

Protected Routes

1. Axios interceptor adds "Authorization: Bearer <token>" header
2. Backend get_current_user() dependency:
 - Extracts token from header
 - Decodes and verifies JWT signature
 - Checks token expiry
 - Fetches user from DB
3. On 401 response, frontend auto-clears token and redirects to /login

10. File Processing

Supported Formats

```
PDF (.pdf) - PyPDFLoader (LangChain) + pdfplumber/PyPDF2 fallback
DOCX (.docx) - Docx2txtLoader (LangChain) + python-docx
XLSX (.xlsx) - openpyxl (manual extraction by sheet/row)
PPTX (.pptx) - python-pptx (slide-by-slide extraction)
TXT (.txt) - TextLoader (LangChain) / direct read
```

Processing Pipeline

1. File validation:
 - Extension check (pdf/docx/xlsx/pptx/txt)
 - Size limit: 50 MB max
 - Empty file check
2. File storage:
 - Saved to ./uploads/ directory
 - Filename: {timestamp}_{uuid}_{original_name}
3. Text extraction (via LangChain loaders)
4. Text splitting:
 - RecursiveCharacterTextSplitter
 - 1000 char chunks, 200 char overlap
 - Smart separators: \n\n > \n > ". " > " " > ""
5. Chunks stored in Embeddings table
6. Pinecone vector storage (optional, fails gracefully)
7. Status updated: pending -> processing -> completed/failed

11. Setup & Installation

Prerequisites

```
Python 3.10+  
Node.js 18+  
npm or yarn
```

Backend Setup

```
# Navigate to project  
cd Universal_Rag_System  
  
# Create virtual environment  
python -m venv venv  
venv\Scripts\activate (Windows)  
source venv/bin/activate (Linux/Mac)  
  
# Install dependencies  
pip install fastapi uvicorn sqlalchemy pydantic[email]  
pip install python-dotenv python-multipart bcrypt PyJWT  
pip install langchain langchain-community langchain-groq  
pip install pypdf docx2txt openpyxl python-pptx pdfplumber  
pip install sentence-transformers  
  
# Create .env file with required variables  
  
# Run backend  
uvicorn backend.main:app --reload --port 8000
```

Frontend Setup

```
cd frontend  
npm install  
npm run dev  
# Runs on http://localhost:5173
```

12. Environment Variables

Create a .env file in the project root with these variables:

```
DATABASE_URL=sqlite:///./test.db
SECRET_KEY=<your-random-secret-key>
GROQ_API_KEY=<your-groq-api-key>
UPLOAD_DIRECTORY=./uploads

# Optional (for Pinecone vector storage)
PINECONE_API_KEY=<your-pinecone-key>
PINECONE_ENVIRONMENT=<your-pinecone-env>
```

Note: Get a free Groq API key at <https://console.groq.com>. Pinecone is optional - the system works without it using local keyword search.

13. Known Limitations & Future Scope

Current Limitations

- Keyword-based search (no semantic/vector search without Pinecone)
- SQLite not suitable for high-concurrency production use
- No rate limiting on API endpoints
- No email verification on registration
- No password reset functionality
- XLSX files processed via manual extraction (no LangChain loader)
- Frontend documentsAPI.process() endpoint has no backend route (minor, non-breaking)
- Pinecone integration attempted but fails gracefully without proper setup

Future Improvements

- Add semantic vector search (Pinecone/ChromaDB/FAISS)
- Migrate to PostgreSQL for production
- Add rate limiting and request throttling
- Implement email verification and password reset
- Add WebSocket for real-time processing status updates
- Support more file formats (CSV, HTML, Markdown)
- Add multi-language support for UI
- Implement document sharing between users
- Add admin dashboard
- Docker containerization for easy deployment